SideQuest: A Social Chatting and Group Hangout Decision-Making Application

Week 2 Progress Report

Tazrean Tasneem, Tahmidatul Ayesha Asha, Zarifa Rahman, Tanzim Ahamad, Soumik Sikder
Course: CSE299 | Group: 02 | Section: 03

***Abstract*— *The report summarizes the progress of SideQuest, a social chatting application designed to support private and group conversations and collaborative hangout planning. The main objective during this development was to connect Flutter frontend, FastAPI backend, and PostgreSQL database. A user database model was configured with SQLAlchemy, and the registration, login, JSON Web Token (JWT) authorization, logout, and database connectivity workflows were tested. Flutter authentication screen was integrated with the backend API, and registration, login, and logout were verified. The integrated system establishes a foundation for the next phase: real-time private and group messaging, persistent message log, and the "Start a SideQuest" collaborative planning feature.*

***Index Terms*— *Flutter, FastAPI, PostgreSQL, SQLAlchemy, JSON Web Token (JWT), authentication, social messaging, WebSocket.*

I. INTRODUCTION

SideQuest is being developed as a social chatting application that enables users to communicate through private and group conversations. Its primary aspect includes features i.e., "Start a SideQuest," is intended to help group members plan hangouts collaboratively and make decisions. After developing the initial frontend prototype and backend authentication foundation, the main objective for Week 2 was to establish a functional connection among Flutter frontend, FastAPI backend, and PostgreSQL database. Progression of SideQuest within Week-2 focused mainly on database configuration, authentication integration, and application testing.

II. DATABASE CONFIGURATION AND INTEGRATION

PostgreSQL was selected as the database management system, and pgAdmin 4 was used to manage and inspect the database. A dedicated database named sidequest_db was created.

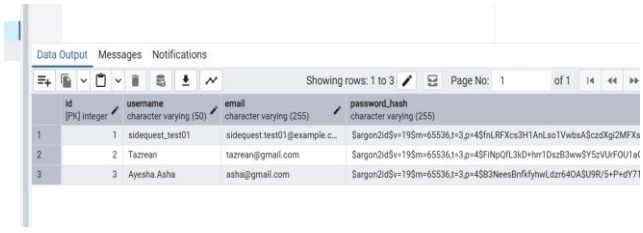


Figure 1.1: database update in pgAdmin 4

SQLAlchemy was used to define the database model for user information and connect it to the database configuration. The application initialized the required table, and the resulting users table was verified in pgAdmin 4.

The users' table contains the ID, username, email address, password hash, account status, and account creation timestamp. Unique constraints on usernames and email addresses help prevent duplicate account registration.

A .env file was created inside the backend directory to store environment-specific settings, including the PostgreSQL connection URL and JWT (JSON web token) signing secret. This separates sensitive configuration from application source code. The .gitignore file was configured to exclude environment secrets and Python virtual environments from version control.

III. BACKEND DEVELOPMENT AND AUTHENTICATION

The backend was developed using Python and FastAPI. Uvicorn was used to run the application locally, while SQLAlchemy handled database interactions. After implementing the backend authentication, the backend was tested:

A. User Registration: The backend accepts registration details, validates the submitted information, hashes the password using Argon2, and stores the resulting user record in PostgreSQL.

B. User Login: The backend verifies submitted credentials against stored user information and returns an access token after successful authentication.

C. JWT Authorization: The generated JWT was entered into Swagger UI's authorization interface and used to test an authenticated request.

D. Logout: The logout operation was tested, and the endpoint returned a successful response.

E. Database Connectivity: The /db-check endpoint returned {database_connected: true}, confirming that the backend could connect to PostgreSQL.

A registration test was executed against existing account in the database to check for account replication. The backend returned HTTP 409 Conflict with a duplicate-account message. These tests confirmed that the backend could communicate with PostgreSQL and perform core authentication operations using actual database records rather than simulated data.

IV. FRONTEND–BACKEND INTEGRATION

An API service file, API_service.dart, was added to the Flutter frontend to organize HTTP requests to the backend. The http package was included in the Flutter dependencies to support communication with API endpoints.

The authentication screen was updated to use the API service for registration and login instead of relying solely on simulated frontend behavior. The frontend collects user credentials and submits them to the backend, which performs authentication against PostgreSQL.

The application was executed in Google Chrome through Flutter's development environment. Registration, login, and logout workflows were tested, which confirmed the integrated authentication flow is successful. This integration establishes the baseline for future developments that require authenticated users and persistent database records.

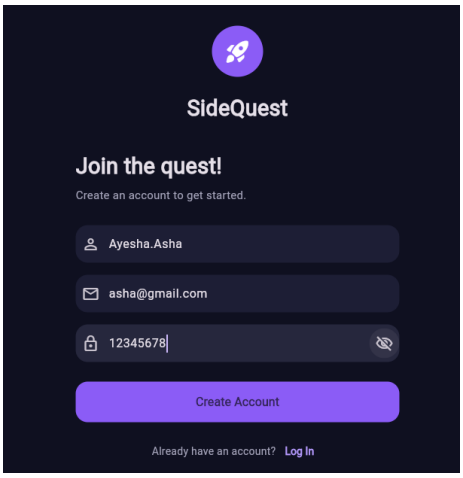


Figure 1.2: Register interface

V. TESTING TOOLS AND APPROACH

A. Visual Studio Code: Used to write and modify Python and Dart code, manage project files, and run development commands.

B. PostgreSQL and pgAdmin 4: Used to create the database, inspect the users table, and verify stored records.

C. FastAPI Swagger UI: Used to execute API requests and examine responses and HTTP status codes.

D. Python virtual environment and pip: Used to manage the backend's Python environment and install required packages.

E. Flutter package manager: Used to resolve and manage frontend dependencies.

The backend was tested independently before the frontend was connected. This helped identify whether an issue originated in the API, database configuration, or frontend integration.

VI. PROBLEMS FACED AND SOLUTIONS

The PostgreSQL connection and environment variables required careful configuration to ensure that the backend accessed the correct database and loaded its settings securely. The .env file was configured, and the connection was verified through the /db-check endpoint.

JWT authorization in Swagger UI required copying the access token into the authorization interface before protected requests could be tested. The authentication workflow was then verified through successful API responses.

During frontend integration, Flutter encountered dependency-resolution errors because the package lockfile contained conflicting content. The dependency configuration was corrected by resolving the Git conflicts in pubspec.yaml and pubspec.lock, then ran flutter pub get to regenerate the lockfile and restore dependency resolution, and the lockfile was regenerated using the Flutter package manager. This restored the frontend dependency-resolution process.

VII. CURRENT PROGRESS AND FUTURE DEVELOPMENT

At the end of this development stage, SideQuest has a Flutter frontend connected to a FastAPI backend and PostgreSQL database. User registration, login, JWT authorization, and logout have been tested successfully. Database connectivity and the persistence of user records have also been verified.

The next development phase will focus on real-time communication. It will require database models for conversations and messages, the corresponding database relationships, and WebSocket communication through FastAPI. The frontend will then exchange messages through the backend rather relying on locally managed or sample conversations.

Further development will include private and group messaging, constant message log, authenticated access to conversations, and the "Start a SideQuest" functionality for collaborative hangout planning.

VIII. CONCLUSION

Week 2 marks a transition from an initial application prototype and backend foundation to a functionally integrated system. The successful connection among Flutter, FastAPI, and PostgreSQL establishes the technical foundation for persistent user accounts and authenticated application workflows. The initial stage of authentication integration also prepares the project for its upcoming objective: implementing real-time private and group messaging. These developments shift SideQuest closer to its intended purpose of combining social communication with collaborative group hangout planning.